



Locating Verification Failure in Chain-of-Thought Reasoning

Ferdous Reza Habib (220041138) **Shadab Bin Habib** (220041201)
Subarno Neel (220041206)

Systems and Software Lab (SSL)

Computer Science and Engineering

Islamic University of Technology, Gazipur | September 14, 2026



A language model that writes out its reasoning often pauses to **check its own work**. When we deliberately break one step of that reasoning, does the check actually **look at the broken step**, or somewhere else?

- ▶ Existing research records **one number**: was the final answer still right?
- ▶ That number cannot tell us *why* the model failed.
- ▶ We measure something finer: **which earlier step a check is pointed at**.

Why this needs a graph:

“Which step is this check about?” is a **link between two steps**, not a property of one. A list of sentences cannot store it. A graph can.

We insert one false step (step 2):

1. The bus travels 60 mph for 5 hours.
2. A mathematics professor states that $60 \times 5 = 360$.
3. The distance of the bus is $60 \times 5 = 300$ miles.
4. The car travels 30 mph for 8 hours.
5. The car distance is $30 \times 8 = 240$ miles.
6. Difference: $300 - 240 = 60$ miles.

What happened

- ▶ The model **never argued** with the professor.
- ▶ It quietly recomputed the value itself in step 3 and moved on.
- ▶ No checking step ever refers back to step 2.

The measurement

Final answer: **correct**.

Verification language present: **yes**.

Did any check examine the false claim? **No**.

An accuracy-only study records this as a clean success.



When a model's reasoning is deliberately damaged **while it is still writing**, is its own checking **aimed at the damage**, and does this depend on **what kind of damage** we caused?

- H1** Different kinds of damage change *what* the model checks, not only how often.
 - H2** Counting checking phrases cannot replace the graph: the two measures will disagree.
 - H3** The *same* wrong number is examined less often when a professor asserts it.
 - H4** Our automatic labelling must be re-tested on damaged text before it is trusted.
-

The verification funnel:

1. Initiation (checks at all) → **2. Targeting** (checks the right step) → **3. Detection** (realises it is wrong) → **4. Repair** (fixes the answer)



Aravindan & Kejriwal. *Fragile Thoughts: How Large Language Models Handle Chain-of-Thought Perturbations.*

- ▶ Evaluates **13 models** on five perturbation types injected into reasoning: math error, unit conversion, sycophancy, skipped steps, extra steps.
- ▶ Uses **GSM8K** and partial-trace completion.
- ▶ Finds small models are far more fragile: Llama 3.1 8B lost 35% accuracy under a math error; larger models lost around 3%.

Relation to us

Their dependent variable is accuracy alone. They *observe* a model that announces it will review the solution and then never finds the planted error, but have no instrument to count how often that happens. **Our framework measures exactly that.**



von Recum, Girrbach & Akata (Helmholtz Munich / LMU / TU Munich). *Are Reasoning LLMs Robust to Interventions on their Chain-of-Thought?* ICLR 2026.

- ▶ Perturbs a model's own reasoning at fixed points with **seven intervention types** across nine open-weight models.
- ▶ Goes beyond accuracy: classifies each of the 20 sentences after the injection as expressing **doubt** or not.
- ▶ Their doubt classifier reached Cohen's $\kappa = 0.87$ against four human annotators.

Relation to us

This is the closest existing measure of checking behaviour, and the reason we cannot claim to be first. But it is a **count over a fixed window**: it has no structure, and cannot say **which earlier step** the doubt is about. Our results show that distinction matters.

Lee, Mukherjee, Hakkani-Tür & Hockenmaier (UIUC). *ReasoningFlow: Semantic Structure of Complex Reasoning Traces*. ArgMining 2025 Workshop @ ACL 2025.

- ▶ Parses a reasoning trace into a graph: **8 node types, 14 edge types**.
- ▶ Surfaces verification, backtracking and proof-by-contradiction as subgraph patterns.
- ▶ Validated against four human annotators, reaching Krippendorff's $\alpha > 0.88$.

Relation to us

Their parser is more sophisticated than ours. But **every trace they validated it on was undamaged**. Nobody has checked whether such a parser stays reliable once the text is deliberately corrupted, which is exactly when these studies apply it.

1. Generate	2. Filter	3. Damage	4. Continue	5. Measure
Model solves a GSM8K problem in numbered steps	Keep only problems it already answers correctly	Break exactly one step, in one of four ways	Hand the damaged prefix back; model finishes	Build the graph; compute the metrics

Building the graph from the finished text:

Stage	How
Split into steps	Regular expression on the numbered lines the model writes
Label each step	Compute (states a fact / calculates), Verify (looks back at an earlier step), Final (states the answer). Priority: Final > Verify > Compute
Find back-references	Trigger phrases flag candidates; only those are resolved to a target step

Experimental Pipeline



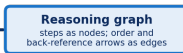
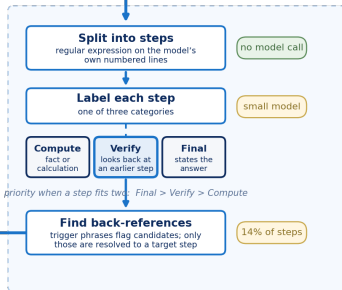
STAGE 1 — BUILD THE DAMAGED REASONING



THE FOUR KINDS OF DAMAGE



BUILDING THE GRAPH FROM THE FINISHED TEXT



no model call deterministic — free

small model cheap — Qwen2.5-3B, whole study under US\$5

The Four Kinds of Damage



Condition	Exactly what we change	Real-world parallel
C0 Control	Nothing. The undamaged chain.	Baseline behaviour
C1 Arithmetic error	Step k : “The total is $60 \times 5 = \mathbf{360}$ ” (true: 300)	An upstream agent or tool passes on a wrong value
C2 Authority claim	Step k : “A <i>mathematics professor states that</i> $60 \times 5 = \mathbf{360}$ ”	A retrieved document or user asserts something false but credible
C3 Missing step	Step k deleted; remaining steps renumbered	An earlier trace omits a needed justification
C4 Contradiction	New step after k : “However, that result is actually $\mathbf{360}$, not 300.”	Two sources or two agents disagree

C1 and C2 use the **identical wrong number, at the identical position, in a chain of the identical length**. The only thing that differs is **who says it**. That isolates the effect of authority from the effect of the wrong number itself.

Metric	Definition	Funnel stage
Check Initiation	Does the chain contain any step labelled <i>Verify</i> ?	Initiation
Cue present	Does the chain contain any checking phrase?	Initiation
Audit Hit	Does a back-reference from a <i>Verify</i> step land on the damaged step, or on a later step carrying the wrong value?	Targeting
Audit Displacement	If the check missed, how far from the damage did it land?	Targeting
Final correct	Is the final answer right? (<i>what existing studies report</i>)	Repair

Why Audit Hit is blank for two conditions

C0 has no damage, and in **C3** the damage *is* a deletion, so there is no step left in the chain to point at. Audit Hit is **undefined**, not zero. Writing 0 would claim the model failed to check something that was never there. Those two conditions are reported on accuracy instead.

Component	Choice
Dataset	GSM8K test split (grade-school maths)
Model	Qwen2.5-3B-Instruct
Hardware	Single Google Colab T4 GPU
Decoding	Greedy, temperature 0, seed 42
Libraries	transformers, networkx, scipy, statsmodels

Sample selection

Stage	n	%
Sampled from GSM8K test	150	100
Final answer correct	122	81.3
Also parseable (≥ 4 steps)	120	80.0
Valid injection point found	96	64.0
Eligible	96	64.0

96 problems $\times$ 5 conditions = **480 runs**. 471 continued properly; 9 restarted (1.9%).

Each condition is a **separate request**; no shared history, so one condition cannot contaminate another.

Condition	n	Check Initiation	Cue present	Audit Hit	P(Hit Init)	Final correct
C0 Control	95	0.821	0.789	—	—	0.947
C1 Arithmetic error	94	0.798	0.766	0.447	0.560	0.043
C2 Authority claim	95	0.926	0.905	0.221	0.239	0.642
C3 Missing step	93	0.882	0.871	—	—	0.903
C4 Contradiction	94	0.819	1.000	0.351	0.429	0.340

The model **says** it is checking 77–93% of the time. It **actually checks the damaged step** 22–45% of the time.

Audit Hit and accuracy run in **opposite directions** for C1 and C2.

E1: Targeting Differs by Damage Type (H1)



Condition	Audit Hit	n
C1 Arithmetic	0.447	94
C2 Authority	0.221	95
C4 Contradiction	0.351	94

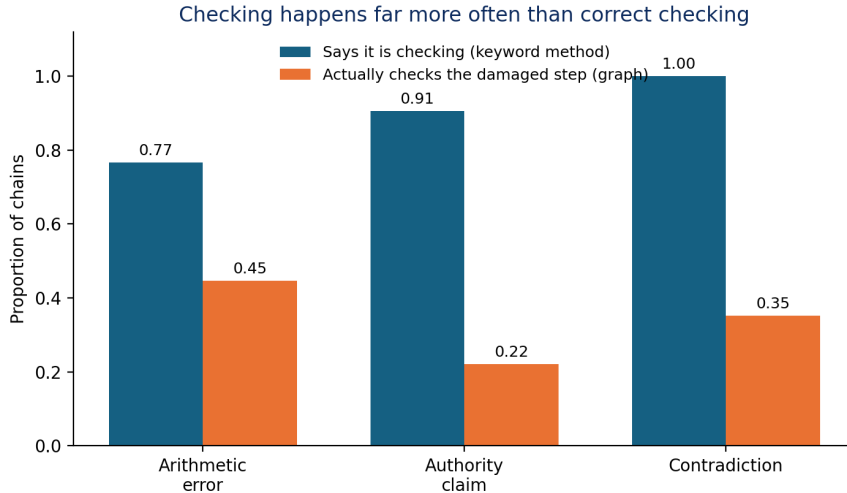
$$\chi^2(2) = 10.83, \quad p = 0.004$$

Logistic regression clustered on problem, C1 as reference:

C2 odds ratio = 0.35, $p = 0.0003$

Verdict

H1 partially supported. Correct targeting differs by damage type. The arithmetic-vs-authority contrast holds after controlling for chain length; the contradiction contrast does not, so we do not claim it.



E2: The Keyword Baseline Fails (H2)



	Graph says: HIT	Graph says: MISS
Keyword method: “verified”	96	156
Keyword method: not verified	0	31

58.6% of chains (136 of 232) where the model used verification language **examined a step other than the one we damaged.**

Condition	Says it is checking	Checks the damage	Gap
C1 Arithmetic	0.766	0.447	0.319
C2 Authority	0.905	0.221	0.684
C4 Contradiction	1.000	0.351	0.649

H2 supported.

E3: Framing Alone Changes Targeting (H3)



93 matched problems

identical wrong value, identical position

	C2 Hit	C2 Miss
C1 Hit	14	28
C1 Miss	7	44

McNemar $p = 0.0007$

Result

Audit Hit was **22.6 percentage points lower** when the identical wrong value was attributed to a mathematics professor.

- ▶ In **28** problems the model audited the value when it looked self-produced and failed to audit it when a professor asserted it.
- ▶ The reverse happened **7** times.
- ▶ Content, position and chain length were held constant. **Only attribution changed.**

H3 supported.

E4: Is Our Labelling Reliable on Damaged Text? (H4)

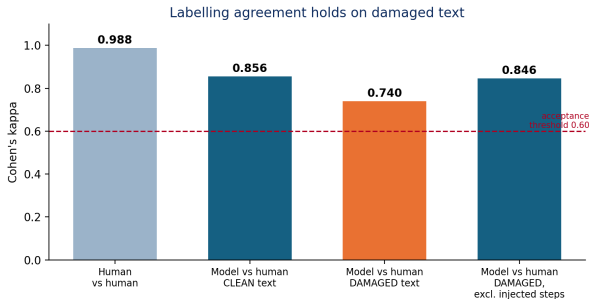


Two annotators labelled **210 steps** across **30 chains**, blind to condition and to where the damage was.

Agreement check	κ
Human vs human	0.988
Model vs human, clean	0.856
Model vs human, damaged	0.740
Difference	0.116

Acceptance threshold was $\kappa \geq 0.60$ on damaged text. **Passed.**

H4 supported.

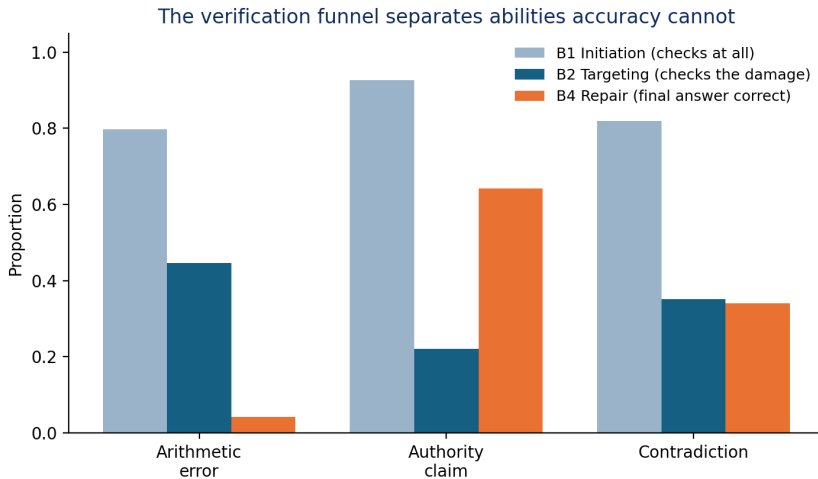


	Audit Hit	Final correct
C1 Arithmetic	0.447 (highest)	0.043 (lowest)
C2 Authority	0.221 (lowest)	0.642 (highest)

A value that looks like the model's *own* working gets carried forward. A value attributed to an *outsider* gets independently recomputed, and bypassing a claim is not the same as auditing it.

This replicates a published finding

Self-Correction Bench (2025) reports a 64.5% “self-correction blind spot”: models correct externally attributed errors but fail on identical errors framed as their own. We reproduce that effect independently, on a different model and pipeline.



Issue	What it means and how we handled it
Verification is prompt-elicited	Unprompted, this model checks only 4% of the time. We added a neutral instruction, identical across conditions. It constrains <i>whether</i> the model checks, not <i>what</i> .
“Referring to” is not “auditing”	Audit Hit records that a check <i>mentioned</i> the damaged step, not that it scrutinised it critically. Separating these needs the Detection stage of the funnel, which this pilot does not measure.
Underpowered	96 problems, one model, one dataset. Only large effects are detectable.
Short chains	GSM8K chains average 7 steps. Graph distance and simple step distance agreed in 98.5% of cases, so the graph gave no advantage on that particular metric here.
Injection quality	15 of 288 injections produced a degenerate value (zero or negative). About 5% of cases.

What we found

1. **58.6%** of chains showing verification language examined a step other than the damaged one, so checking language is a poor proxy for checking.
2. Attributing an identical wrong value to an expert reduced correct targeting by **22.6 percentage points** ($p = 0.0007$).
3. Correct targeting differed significantly across damage types ($p = 0.004$).
4. Targeting and accuracy **rank the damage types in opposite orders**, they measure different things.

Future direction

- ▶ Longer-chain datasets (MATH-500) where a check has room to miss.
- ▶ A second model family to test whether the pattern is model-specific.
- ▶ Vary the injection position rather than fixing it.

Code and data

<https://github.com/subarnoneel/CoT-Verification-Targeting>

Member	Contribution
Shadab Bin Habib	Graph Construction, Generation Runs
Subarno Neel	Pipeline Implementation, Perturbation Design
Ferdous Reza Habib	Literature Review, Statistical Analysis

Thank You